



PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

CampusNavX: A Graph-Based Campus Navigation System for Indoor, Outdoor, Accessible, and Emergency Routing Using BFS and Advanced Data Structures

<i>Team Name:</i> <i>Team #</i>	<i>The Pathfinders</i> <i>DSCPP-III-2026-T377</i>
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	<i>Ambika – 2510380114</i> <i>2510380114@geu.ac.in</i> 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	<i>Tripathi Shivang –</i> <i>2510012567</i> <i>2510012567@geu.ac.in</i> 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	<i>Mohd Uvesh –</i> <i>2510210310</i> <i>2510210310@geu.ac.in</i> 

PHASE-II PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

(Brief restatement of your project's main goal. Max 300 words).

CampusNavX is a navigation system that uses graphs to help people move around a university campus. It deals with paths, accessible routes and emergency exits. The system is written in C++ so it can work quickly and provide answers fast. All the information about buildings, rooms, labs, walkways and other things on campus is kept in a graph. In this graph each place is a vertex and the connections between them are edges. To find a route CampusNavX uses BFS and DFS. Then Dijkstras algorithm to find the shortest path. It depends on data structures like queues for linked lists and stacks for moving through the graph. It uses adjacency lists to keep the graph structure organized. Users can look up a location, block or unblock paths see routes and calculate the distance between two places.

The system can be expanded to add a user interface. This would allow people to view results clearly and learn how powerful graph-based solutions and the BFS, DFS and Dijkstra algorithms can be.

CampusNavX is not a tool it's a proof of concept. It shows how data structures and algorithms can work together to create a real-world navigation system, for a university campus.

Updated Project Approach and Architecture (2 pts)

(Describe your current approach, including system design, communication protocols, libraries used, etc.

Max 300 words).

CampusNavX is being developed as a graph-based campus navigation system. It supports both indoor navigation. The system helps users plan routes and provides guidance for emergency exits.

System Design: The backend is built in C++ with a structure. Campus locations are shown as graph vertices. Pathways between these locations are represented as edges. Each edge holds information like distance. Whether the path is accessible. Separate modules manage graph data, navigation algorithms, indoor locations, emergency routing and file handling. Breadth-First Search (BFS) is used for finding routes with the steps. Depth-First Search (DFS) is used for exploration of the graph. Dijkstra's algorithm finds the path based on distance. Accessibility features take into account any restrictions on pathways. Emergency navigation identifies the fastest exits during an emergency.

Data Management: The system stores campus locations, pathways, indoor spaces and route history in CSV and text files. This makes it easy to load campus data and keep track of navigation paths. The system can also. Unblock pathways to reflect changes in route availability.

Communication: The graphical user interface is being built using HTML, CSS and JavaScript. A communication layer will be added to link the frontend with the C++ backend. This allows users to pick destinations, request routes and see navigation results directly in a web browser.

Libraries and Tools: The backend uses C++ libraries for data structures, input/output and file operations. The frontend uses HTML, CSS and JavaScript. No separate UI framework is needed. The project is compiled using a C++ compiler. CMake and batch scripts help manage the build process.

Current Status: The core backend and navigation modules are complete. Work is still going on for the GUI, frontend-backend integration and full end-, to-end testing.

Tasks Completed (7 pts)

(Describe the main tasks that have been assigned and already completed. Max 250 words).

Task Completed	Team Member
Member 1: Backend and Data Structures • Implemented graph data structures. • Developed BFS, DFS, and Dijkstra algorithms. • Managed campus location and pathway data. • Added pathway blocking and route history.	Ambika
Member 2: Navigation and Route Management • Developed indoor navigation. • Implemented accessible and emergency routing.	Uvesh
Member 3: GUI, Integration, and Testing • Worked on the HTML, CSS, and JavaScript interface. • Assisted with frontend-backend integration. • Prepared route and algorithm test cases.	Shivang Tripathi

Challenges/Roadblocks (7 pts)

(Describe the challenges that you have faced or are facing so far and how you plan to solve them. Max 300 words).

During the development of CampusNavX, our team has encountered numerous challenges with graph construction, route planning, data management, and system integration.

First, we have encountered many difficulties with constructing a realistic and comprehensive campus graph. An ill-defined graph results in unreasonable suggestions of pathways that do not exist. We propose to solve this challenge by verifying the correctness of the campus location and pathway data stored in our CSV files, as well as testing various source-destination queries.

Furthermore, we have faced many issues with the integration of modules, such as those concerning outdoor, indoor, accessible, and emergency pathways. We also experienced discrepancies in function declarations, header files, and implementations, which resulted in compilation and linking errors. We plan on overcoming these issues by developing a modular project and standardizing function interfaces, as well as frequently compiling and testing the integrated project.

The challenge of implementing an efficient route planning mechanism constitutes another issue that we faced during the development of our application. The system must accurately avoid blocked pathways and suggest the most appropriate way of navigation, which may include a combination of different transportations. We intend to address this challenge by extensively testing the breadth-first search algorithm and implementing relevant restrictions for accessible and emergency navigation.

Indoor navigation presents with a set of challenges related to the representation of rooms, elevators, and connections between different floors. We plan on improving the validity of our indoor location data and testing connectivity between various rooms and floors.

Lastly, we have encountered the challenge of designing and implementing a convenient graphical interface without compromising the data structures and algorithms used in our application. We intend to first design the graphical user interface and equip it with the necessary functions, such as highlighting of the shortest or most accessible pathways, as well as the implementation of a campus map and visualization of the breadth-first search progress.

Tasks Pending (7 pts)

(Describe the main tasks that you still need to complete. Max 250 words).

Task Pending	Team Member (to complete the task)
GUI Development • Complete the HTML, CSS, and JavaScript interface. • Add navigation controls, route results, and validation messages.	Shivang Tripathi
Backend Integration • Connect the GUI with the existing C++ backend. • Verify route calculations and data updates.	Ambika
Testing and Documentation • Test routes, algorithms, blocked paths, and emergency navigation. • Finalize documentation and prepare the project demonstration.	Uvesh

Project Outcome/Deliverables (2 pts)

(Describe what are the key outcomes / deliverables of the project. Max 200 words).

The end result of the project is a graph-based campus navigation system implemented in C++ that utilizes the data structures and algorithms. The final outcome will consist of the following components:

Outdoor Navigation: a graph-based finding the shortest path between campus buildings and facilities using the Breadth-First Search algorithm Indoor Navigation: a graph-based finding the shortest path between classrooms, laboratories, and other locations on different floors Accessible Navigation: finding the shortest path between classrooms, laboratories, and other locations avoiding stairs Emergency Navigation: finding the shortest path between classrooms, laboratories, and other locations avoiding blocked pathways identifying open emergency exits Pathway Management: blocking and unblocking pathways, and updating pathway information Location Search: searching for facilities by name, type, or a floor Route History: a stack-based history of the route taken to help the user navigate back Data Management: a CSV file database for storing the information about the campus locations, buildings, and pathways Graphical User Interface: a graphical user interface with a campus map, highlighted paths, indoor navigation, and visualizations of the Breadth-First-Search algorithm

The project is expected to fully incorporate graphs, queues, linked lists, and stacks data structures, and algorithms into creating the application. The final product will be an interactive and extensible prototype that can be updated with verified campus maps and the actual state of the pathways.

Progress Overview (2 pts)

(Summarize how much of the project is done, what's behind schedule, what's ahead of schedule. Max 200 words.)

CampusNavX, a graph-based campus navigation tool has made strong progress on the backend side during the past week. The C++ code has been set up with all the needed data structures and algorithms. These include Breadth-First Search (BFS) Depth-First Search (DFS) Dijkstra's algorithm, indoor navigation features, accessible route finding, emergency exit navigation, pathways and CSV processing. The backend was reported to be working and a successful checklist test confirmed its stability.

Far the backend has been completed along with campus data files and indoor data files. Algorithms for creating routes have been. The accessibility features are in place. Emergency navigation is also functional and route history tracking has been implemented.

Frontend development is underway using HTML, CSS and JavaScript. A communication layer between the browser and the C++ back-end is active. However the frontend files were empty at the start so the user interface still needs to be built and loaded properly on a Windows machine.

There is a delay in integrating and testing the graph-based campus navigation GUI. Also it is unclear whether the new folder with the backend code contains everything that is needed. This step is crucial before moving On the side finishing the backend early has given the project an advantage. It removed the need to spend time on building algorithms from scratch allowing the team to focus more on the frontend and user experience.

Overall the project is now, in the stages of completion and demonstration. Most of the work has gone into developing the backend. No official timeline was ever set, so comparing what was planned to what has been achieved is based on judgment.

Codebase Information (2 pts)

(Repository link, branch, and information about important commits.)

Repository Link: <https://github.com/ambika-02p/DSA-Project-Based-Learning-with-Cpp>

Branch: 'main'

Commits:

Initial Project Setup: We have set up the C++ project structure and divided the source files into algorithms, data structures, navigation and data folders.

Graph Algorithms Implementation: We have implemented BFS, DFS and Dijkstra's algorithm to support campus route planning.

Indoor Navigation: We have added functionality for location tracking and pathway handling allowing navigation between rooms and buildings.

Accessibility and Emergency Features: We have included route planning, pathway blocking and unblocking and emergency exit navigation.

Data Handling: We have integrated CSV files to store campus locations, pathways, indoor locations and route history.

GUI Development: We have started building the HTML, CSS and JavaScript interface. Created the communication layer between the frontend and the C++ backend.

Status: The backend development is mostly complete. We are still working on integrating the GUI and we are performing end-to-end testing and finishing up on the validation.

Testing and Validation Status (2 pts)

(Provide information about any tests conducted)

Test Type	Status (Pass/Fail)	Notes
Route Testing	Pass	Checking source, destination, and route accuracy.
Algorithm Testing	Pass	Testing BFS, DFS, and Dijkstra results.
Blocked Route Testing	Fail	Testing alternate routes around blocked paths.
Accessibility Testing	Pass	Test routes that avoid inaccessible paths.
Indoor Navigation Testing	Fail (improvement in progress)	Testing routes between indoor locations.
Emergency Route Testing	In Progress	Checking nearest accessible exit selection.

Deliverables Progress (2 pts)

(Summarize the current status of all key project deliverables mentioned earlier. Indicate whether each deliverable is completed, in progress, or pending.)

Completed Deliverables: The C++ backend for the campus navigation application CampusNavX has been finished. It includes a graph data structure and implements algorithms such as Breadth-First-Search Depth-First-Search and Dijkstras Shortest Path algorithm. Route calculation is supported, including navigation, accessibility routing and emergency exit routing. The backend can. Unblock pathways, read and write data in CSV format and keeps track of calculation history.

Deliverables in Progress: The graphic user interface of the application is being built using HTML, CSS and JavaScript. The connection between the frontend and the backend is being tested locally on the Windows operating system. Project documentation has been. Includes the abstract technical approach, repository details and progress report.

Pending Deliverables: End-to-end testing of the application needs to be done to make sure all features work properly. This will include testing route calculation, indoor navigation, accessibility routing, emergency exit routing and pathway blocking and unblocking through the graphical user interface. The code in the GitHub repository must be updated. The final documentation and project presentation need to be prepared for submission.

Overall Status: The project is going well. The backend of CampusNavX is stable and fully functional. The focus now is, on finishing the user interface making sure it works reliably and testing all features thoroughly. All functionalities must be confirmed before the final version of the project is submitted.